

ملخص مراحل تطوير مشروع متجر إلكتروني

1. إنشاء المشروع وتعريف نماذج البيانات
 - إنشاء مشروع متجر إلكتروني باستخدام Django وربطه بقاعدة بيانات PostgreSQL.
 - تعريف الموديلات الأساسية: التصنيفات، المنتجات، الطلبات وعناصر الطلب.
 - إنشاء وتطبيق Migrations، ثم إضافة بيانات تجريبية وحساب مدير.
2. إنشاء GraphQL API
 - إضافة Graphene-Django وبناء GraphQL API للمتجر.
 - تنفيذ Queries وMutations للمنتجات والتصنيفات والطلبات، مع الفلترة وRelay Global IDs.
 - تطبيق عمليات الطلبات مع حصر ظهور طلبات الزبون بطلباته الخاصة.
3. Authentication وAuthorization
 - تنفيذ التسجيل وتسجيل الدخول باستخدام JWT مع Access Token وRefresh Token.
 - إضافة Refresh Token Rotation وLogout وإبطال التوكنات.
 - استخدام جداول في قاعدة البيانات لحالة التوكنات في البداية، ثم استبدالها لاحقاً بتخزين الحالة في Redis.
 - إنشاء Groups وPermissions لأدوار Customer وStaff مع صلاحيات الملكية للطلبات.
4. إضافة Redis والـ Caching
 - استخدام Redis لتخزين حالة إبطال التوكنات وإصداراتها.
 - استخدام Redis Database منفصلة لعمل Cache للمنتجات والتصنيفات.
 - إضافة آلية Cache Invalidation عند تعديل أو حذف المنتجات والتصنيفات.
5. إضافة REST API إلى جانب GraphQL
 - إضافة Django REST Framework مع الإبقاء على GraphQL؛ الواجهتان تعملان معاً.
 - تنفيذ REST endpoints للمصادقة والمنتجات والتصنيفات والطلبات.
 - إضافة Pagination وSearch وOrdering وFiltering، مع إعادة استخدام نفس Services وPermissions وJWT وCache.
6. إنشاء Postman Collections
 - إنشاء Collections منفصلة لاختبار GraphQL وREST.
7. استيراد وتصدير CSV
 - إضافة Import/Export للمنتجات وExport للطلبات للمستخدمين المصرح لهم.
 - التحقق من كامل الملف قبل الحفظ واستخدام Transaction Atomic لمنع الاستيراد الجزئي عند وجود أخطاء.
8. Rate Limiting
 - إضافة Rate Limiting لإنشاء الطلبات عبر REST وGraphQL.
 - تحديد عدد الطلبات ضمن نافذة زمنية لكل IP ولكل مستخدم بشكل مستقل.
 - استخدام Redis Sorted Sets وLua Script لتنفيذ التحقق بشكل Atomic ومنع تجاوز الحد مع الطلبات المتزامنة.
9. Celery للمهام – استخدام Redis كـ Message Broker أولاً
 - إضافة Celery Worker وCelery Beat لتنفيذ المهام الخلفية والمجدولة.
 - استخدام Redis في البداية كـ Message Broker لنقل رسائل مهام Celery، مع Redis Database أخرى كـ Result Backend.
 - تنفيذ مهمة دورية لإلغاء الطلبات التي بقيت PENDING بعد انتهاء المهلة المحددة.

10. استبدال Redis بـ RabbitMQ كـ Message Broker

- إضافة RabbitMQ واستبدال Redis في دور Message Broker الخاص بـ Celery.
- Celery Beat يرسل المهام المجدولة إلى RabbitMQ، و Celery Worker يستهلك الرسائل وينفذ المهام.
- الإبقاء على Redis لاستخدامه في JWT State و Caching و Rate Limiting و Celery Result و Backend.

11. مهام تأكيد الطلبات بالخلفية

- إرسال مهمة تأكيد الطلب بعد نجاح Database Transaction باستخدام transaction.on_commit.
- RabbitMQ ينقل رسالة المهمة، و Celery Worker ينفذ مهمة التأكيد بشكل غير متزامن.

12. Docker و Docker Compose

- Containerization للتطبيق والخدمات المساندة.
- تشغيل وتنسيق Django/Gunicorn و PostgreSQL و Redis و RabbitMQ و Celery Worker و Celery Beat و Nginx عبر Compose.
- إضافة Healthchecks و Persistent Volumes وإعدادات مشتركة بين خدمات التطبيق.

13. إضافة Nginx

- إضافة Nginx كمدخل HTTP رئيسي أمام Gunicorn/Django.
- تمرير REST و GraphQL و Admin إلى Gunicorn، وخدمة Static Files مباشرة من Nginx.
- إضافة Forwarded Headers و Body Size Limit و Health Check و Timeout مناسب لطلبات الـ AI المحلية.

14. إدارة الأسرار محلياً

- نقل القيم الحساسة من الإعدادات العادية إلى Secret Files محلية.
- تقييد صلاحيات قراءة وكتابة ملفات الأسرار وعدم رفعها إلى Git.
- استخدام Docker Compose لعمل Mount للأسرار داخل الخدمات التي تحتاجها فقط.

15. استخدام Infisical لإدارة الأسرار

- استخدام Infisical Cloud لاحقاً كمصدر رئيسي لإدارة الأسرار بدلاً من إدارتها يدوياً.
- إنشاء Project و Environment و Machine Identity بصلاحيات قراءة فقط.
- استخدام Infisical CLI و Script لجلب الأسرار ووضعها في نفس ملفات Docker Secrets المستخدمة سابقاً دون طباعة قيمها.

16. AI Search محلي

- إضافة بحث ذكي للمنتجات باستخدام Ollama ونموذج Qwen محلياً دون الحاجة إلى API مدفوع.
- تحويل طلب المستخدم باللغة الطبيعية إلى Filters منظمة مثل كلمات البحث والتصنيف والحد الأدنى/الأعلى للسعر والترتيب.